OpenCV in Python — Complete Notes A to Z

1. What is OpenCV?

OpenCV (Open Source Computer Vision Library) is a Python library used to process images and videos, detect objects, work with cameras, and build computer vision applications.

Installation:

```
pip install opencv-python
```

Import:

```
import cv2
```

Here, `cv2` is the module name used to access OpenCV functions.

---

2. Read and Display an Image

```
import cv2

img = cv2.imread("image.jpg")

cv2.imshow("My Image", img)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

Explanation

- `cv2.imread("image.jpg")` — reads an image from the specified path.
- `img` — stores the image as a NumPy array.
- `cv2.imshow("My Image", img)` — displays the image in a window.
- `cv2.waitKey(0)` — waits until a key is pressed.
- `cv2.destroyAllWindows()` — closes all OpenCV windows.

Important: If the image path is incorrect, `imread()` returns `None`.

```
if img is None:
    print("Image not found")
else:
    cv2.imshow("Image", img)
```

```
        cv2.waitKey(0)
        cv2.destroyAllWindows()
```

---

3. Image Shape, Height, Width and Channels

```
import cv2

img = cv2.imread("image.jpg")

print(img.shape)
print(img.ndim)
print(img.size)
print(img.dtype)

height, width, channels = img.shape

print("Height:", height)
print("Width:", width)
print("Channels:", channels)
```

Meaning

- `img.shape` — returns `(height, width, channels)` for a normal color image.
- `img.ndim` — number of dimensions, usually `3` for a color image.
- `img.size` — total number of array elements.
- `img.dtype` — data type, commonly `uint8`.
- `height` — number of rows.
- `width` — number of columns.
- `channels` — usually `3` for a BGR image.

Example: If `img.shape` is `(400, 600, 3)`, the image is 400 pixels high, 600 pixels wide, and has 3 color channels.

For a grayscale image, the shape is usually `(height, width)`, without a channel dimension.

---

4. BGR to RGB Conversion

OpenCV reads color images in **BGR order**, whereas many other libraries use RGB.

```
import cv2

img = cv2.imread("image.jpg")

rgb_img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

cv2.imshow("BGR Image", img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Important functions

```
cv2.COLOR_BGR2RGB
cv2.COLOR_BGR2GRAY
cv2.COLOR_RGB2BGR
```

Examples:

- `COLOR_BGR2RGB` — converts BGR to RGB.
- `COLOR_BGR2GRAY` — converts a color image to grayscale.
- `COLOR_RGB2BGR` — converts RGB to BGR.

Remember: `cv2.imshow()` normally displays an OpenCV image correctly in BGR format. RGB conversion is useful when working with libraries such as Matplotlib that expect RGB.

---

5. Resize an Image

A. Fixed Width and Height

```
import cv2

img = cv2.imread("image.jpg")

resized_img = cv2.resize(img, (300, 200))

cv2.imshow("Resized Image", resized_img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Syntax:

```
cv2.resize(image, (width, height))
```

Remember: The resize tuple is `(width, height)`, not `(height, width)`.

B. Resize by Percentage

```
import cv2

img = cv2.imread("image.jpg")

height, width = img.shape[:2]

new_width = int(width * 0.5)
new_height = int(height * 0.5)

resized_img = cv2.resize(
    img,
    (new_width, new_height)
)

cv2.imshow("50 Percent Image", resized_img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Here, `0.5` means 50% of the original dimensions.

For 25%:

```
new_width = int(width * 0.25)
new_height = int(height * 0.25)
```

`int()` converts the calculated dimensions into integers because pixel dimensions must be whole numbers.

---

6. Draw a Line — `cv2.line()`

```
cv2.line(img, (50, 50), (300, 200), (0, 255, 0), 3)
```

Syntax:

```
cv2.line(image, pt1, pt2, color, thickness)
```

- `image` — image on which the line is drawn.

- `pt1` — starting coordinate `(x1, y1)`.
- `pt2` — ending coordinate `(x2, y2)`.
- `color` — color in BGR format.
- `thickness` — line thickness in pixels.

Complete example

```
import cv2

img = cv2.imread("image.jpg")

cv2.line(
    img,
    (50, 50),
    (300, 200),
    (0, 255, 0),
    3
)

cv2.imshow("Line Image", img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Common BGR Colors

```
blue = (255, 0, 0)
green = (0, 255, 0)
red = (0, 0, 255)
white = (255, 255, 255)
black = (0, 0, 0)
```

Remember: The origin `(0, 0)` is at the top-left corner. The x-coordinate increases to the right, and the y-coordinate increases downward.

---

7. Write Text on an Image — `cv2.putText()`

```
cv2.putText(
    img,
    "Hello World",
    (50, 100),
    cv2.FONT_HERSHEY_SIMPLEX,
```

```
        1,
        (0, 0, 255),
        2
    )
```

Syntax:

```
    cv2.putText(
        image,
        text,
        org,
        fontFace,
        fontScale,
        color,
        thickness
    )
```

- `image` — target image.
- `text` — text to display.
- `org` — bottom-left position of the text baseline.
- `fontFace` — font style.
- `fontScale` — text size.
- `color` — text color in BGR.
- `thickness` — thickness of the text strokes.

Common fonts

```
    cv2.FONT_HERSHEY_SIMPLEX
    cv2.FONT_HERSHEY_PLAIN
    cv2.FONT_HERSHEY_DUPLEX
    cv2.FONT_HERSHEY_COMPLEX
```

Complete example

```
    import cv2

    img = cv2.imread("image.jpg")

    cv2.putText(
        img,
        "OpenCV",
```

```
        (50, 100),
        cv2.FONT_HERSHEY_SIMPLEX,
        2,
        (255, 0, 0),
        3
    )

    cv2.imshow("Text Image", img)
    cv2.waitKey(0)
    cv2.destroyAllWindows()
```

---

8. Draw a Rectangle and Circle

Rectangle

```
cv2.rectangle(img, (50, 50), (250, 200), (0, 255, 0), 3)
```

Syntax:

```
cv2.rectangle(image, pt1, pt2, color, thickness)
```

- `pt1` — top-left corner.
- `pt2` — bottom-right corner.
- Positive thickness draws an outline.
- `thickness=-1` fills the rectangle.

Circle

```
cv2.circle(img, (200, 150), 50, (0, 0, 255), 3)
```

Syntax:

```
cv2.circle(image, center, radius, color, thickness)
```

- `center` — center coordinate.
- `radius` — radius in pixels.
- `thickness=-1` fills the circle.

These functions modify the image array directly.

---

9. Webcam / Live Video Capture

This is one of the most important topics from our class.

```
import cv2

cap = cv2.VideoCapture(0)

while True:
    ret, frame = cap.read()

    if not ret:
        print("Failed to capture frame")
        break

    cv2.imshow("Webcam", frame)

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

Line-by-line explanation

1. Open the webcam

```
cap = cv2.VideoCapture(0)
```

- `VideoCapture()` opens a video source.
- `0` usually refers to the default webcam.
- `1` may refer to another connected camera.

2. Run an infinite loop

```
while True:
```

The camera continuously provides new frames, so we use a loop.

3. Read a frame

```
ret, frame = cap.read()
```

- `ret` — Boolean indicating whether the frame was read successfully.
- `frame` — the captured image.

4. Check for failure

```
    if not ret:
        break
```

Stops the loop if the camera fails or no frame is available.

5. Display the frame

```
    cv2.imshow("Webcam", frame)
```

Shows the current frame in a window. Repeating this process creates the live video effect.

6. Detect the Q key

```
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break
```

- `waitKey(1)` — waits approximately 1 millisecond before checking for a key, allowing the display to update.
- `ord('q')` — returns the character code for `q`.
- `& 0xFF` — extracts the lower 8 bits of the key code, a common OpenCV pattern.
- `break` — exits the loop when Q is pressed.

7. Release resources

```
    cap.release()
    cv2.destroyAllWindows()
```

- `release()` frees the camera or video resource.
- `destroyAllWindows()` closes OpenCV display windows.

---

10. Capture a Video from a File

The same logic works with a saved video.

```
    import cv2

    cap = cv2.VideoCapture("video.mp4")

    while cap.isOpened():
        ret, frame = cap.read()
```

```
        if not ret:
            break

        cv2.imshow("Video Player", frame)

        if cv2.waitKey(25) & 0xFF == ord('q'):
            break

    cap.release()
    cv2.destroyAllWindows()
```

Here, `25` milliseconds is a delay between displayed frames. The video playback speed depends on this delay and the source video's frame rate.

For webcam capture, `VideoCapture(0)` opens a camera; for a saved video, pass its file path.

---

11. Important Differences to Remember

Function	Purpose
`cv2.imread()`	Read an image
`cv2.imshow()`	Display an image or frame
`cv2.waitKey(0)`	Wait for a key press
`cv2.waitKey(1)`	Short delay for live display
`cv2.resize()`	Resize an image

`cv2.cvtColor()`	Convert color spaces
`cv2.line()`	Draw a line
`cv2.putText()`	Write text
`cv2.rectangle()`	Draw a rectangle
`cv2.circle()`	Draw a circle
`cv2.VideoCapture()`	Open a camera or video
`cap.read()`	Read the next frame
`cap.release()`	Release the video resource
`cv2.destroyAllWindows()`	Close OpenCV windows

---

12. Common Mistakes

1. **Wrong image path:** `imread()` returns `None`; check the result before accessing `shape`.
2. **Wrong resize order:** `cv2.resize(img, (width, height))` expects width first.
3. **Confusing BGR and RGB:** OpenCV's default image reading order is BGR.
4. **Forgetting `waitKey()`:** The window may not behave as expected without the event-processing delay.
5. **Forgetting `release()`:** The webcam resource may remain occupied.
6. **Incorrect function capitalization:** Write `cv2.VideoCapture(0)`, not `cv2.videoCapture(0)`.
7. **Using the wrong color tuple:** `(0, 0, 255)` is red in BGR.
8. **Not checking `ret`:** A failed frame read should not be passed to `imshow()`.

---

13. Practice Questions

1. Read an image and print its height, width, channels and data type.
2. Resize an image to 50% of its original dimensions.
3. Convert a BGR image to grayscale.
4. Draw a red line and a green rectangle on the same image.
5. Write your name on an image using `cv2.putText()`.
6. Open a webcam and exit when Q is pressed.
7. Open a saved video file and display its frames.
8. Explain why `cap.read()` returns two values.
9. Explain the difference between `waitKey(0)` and `waitKey(1)`.
10. Explain why `cap.release()` is important.

Final revision: OpenCV images are represented as NumPy arrays. You can read, inspect, resize and modify those arrays. Video capture repeatedly reads image frames, displays them, and releases the resources when finished.